

Protocoles de Transport : *Contrôle de Congestion avec TCP*

Rémi Badonnel
Slides d'Isabelle Chrisment



1

Contrôle de congestion et TCP

- **Contrôle de congestion avec TCP**
 - TCP : quelques rappels
 - Congestion Internet
 - TCP Tahoe
 - TCP Reno
 - RTT et TCP
 - TCP Vegas
 - Équité TCP

2

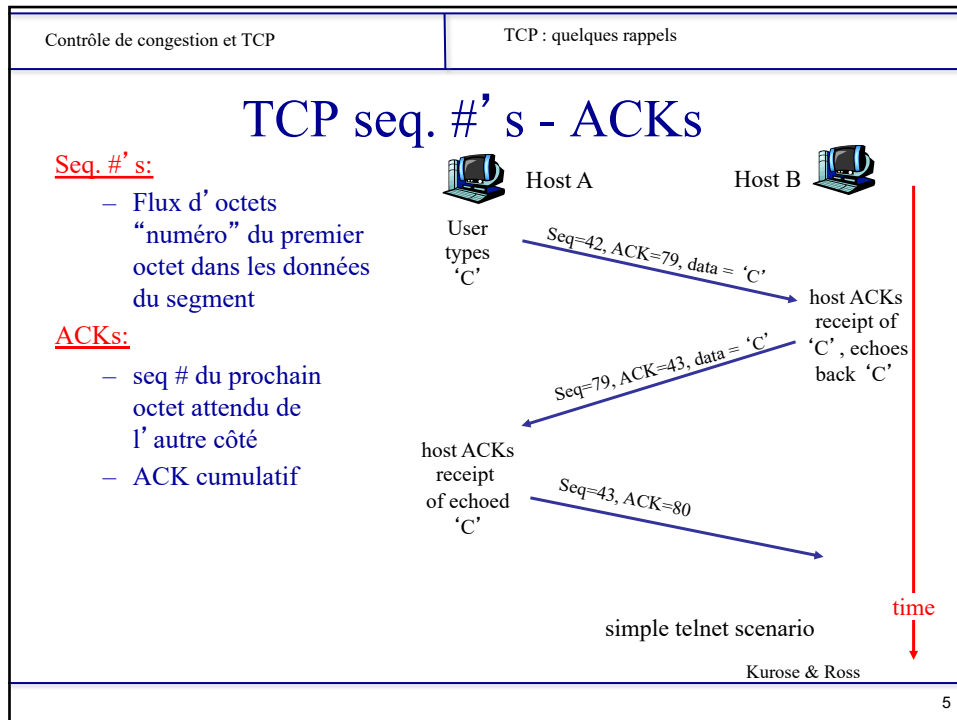
2

Contrôle de congestion et TCP	TCP : quelques rappels
TCP : quelques rappels RFCs: 793, 1122, 1323, 2018, 2581 - Point-à-point - Orienté connexion - Fiable et ordonné - Pas de frontière de message - Full duplex - Flux contrôlé	
3	

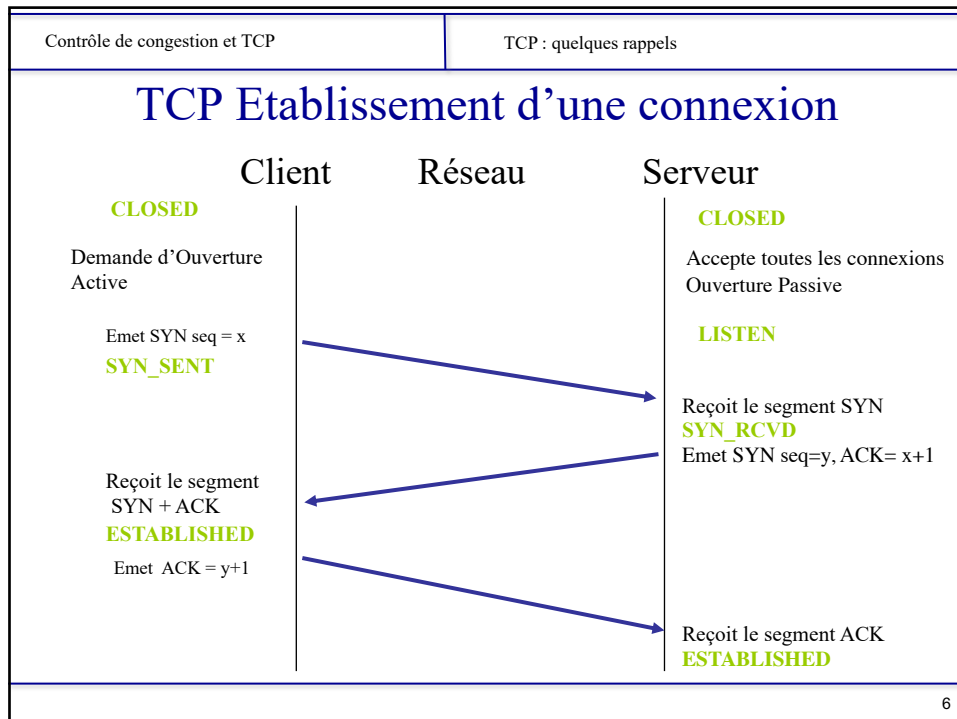
3

Contrôle de congestion et TCP	TCP : quelques rappels																																																																																																																																												
Structure d'un segment																																																																																																																																													
URG: urgent data (generally not used) ACK: ACK # valid PSH: push data now (generally not used) RST, SYN, FIN: connection estab (setup, terminaison commands) Internet checksum (as in UDP) 32 bits <table border="1" style="margin: auto; border-collapse: collapse;"> <tr> <td colspan="10" style="padding: 5px;">source port #</td> <td colspan="10" style="padding: 5px;">dest port #</td> </tr> <tr> <td colspan="20" style="padding: 5px;">sequence number</td> </tr> <tr> <td colspan="20" style="padding: 5px;">acknowledgement number</td> </tr> <tr> <td style="padding: 5px;">head len</td> <td style="padding: 5px;">not used</td> <td style="padding: 5px;">U</td> <td style="padding: 5px;">A</td> <td style="padding: 5px;">P</td> <td style="padding: 5px;">R</td> <td style="padding: 5px;">S</td> <td style="padding: 5px;">F</td> <td colspan="12" style="padding: 5px;">rcvr window size</td> </tr> <tr> <td colspan="10" style="padding: 5px;">checksum</td> <td colspan="10" style="padding: 5px;">ptr urgent data</td> </tr> <tr> <td colspan="20" style="padding: 5px;">Options (variable length)</td> </tr> <tr> <td colspan="20" style="padding: 5px;">application data (variable length)</td> </tr> </table> counting by bytes of data (not segments!) # bytes rcvr willing to accept		source port #										dest port #										sequence number																				acknowledgement number																				head len	not used	U	A	P	R	S	F	rcvr window size												checksum										ptr urgent data										Options (variable length)																				application data (variable length)																			
source port #										dest port #																																																																																																																																			
sequence number																																																																																																																																													
acknowledgement number																																																																																																																																													
head len	not used	U	A	P	R	S	F	rcvr window size																																																																																																																																					
checksum										ptr urgent data																																																																																																																																			
Options (variable length)																																																																																																																																													
application data (variable length)																																																																																																																																													
Kurose & Ross																																																																																																																																													
4																																																																																																																																													

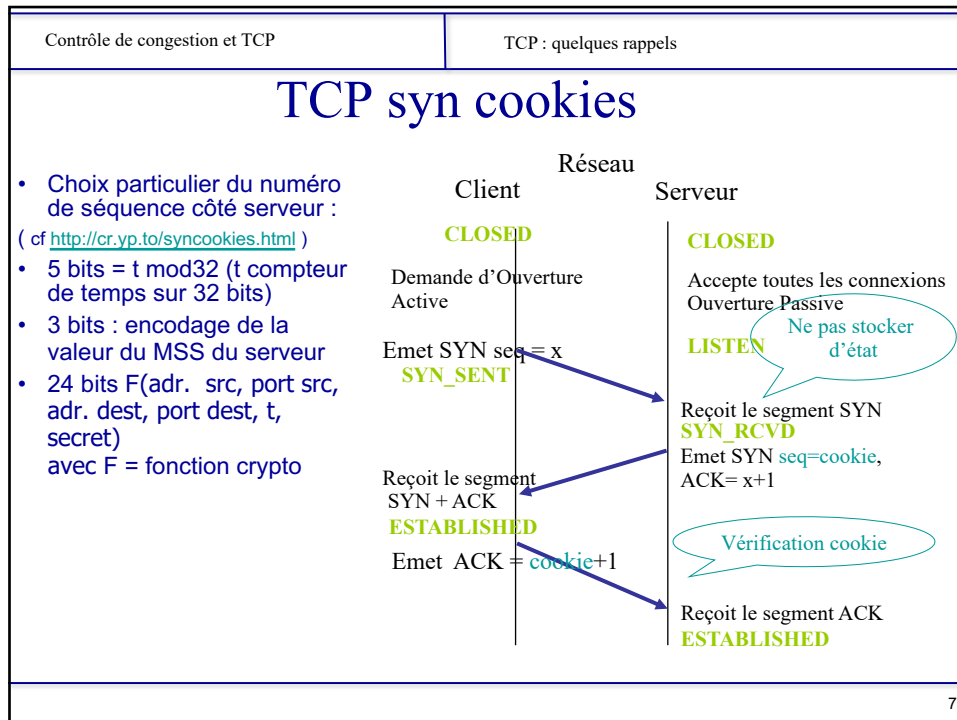
4



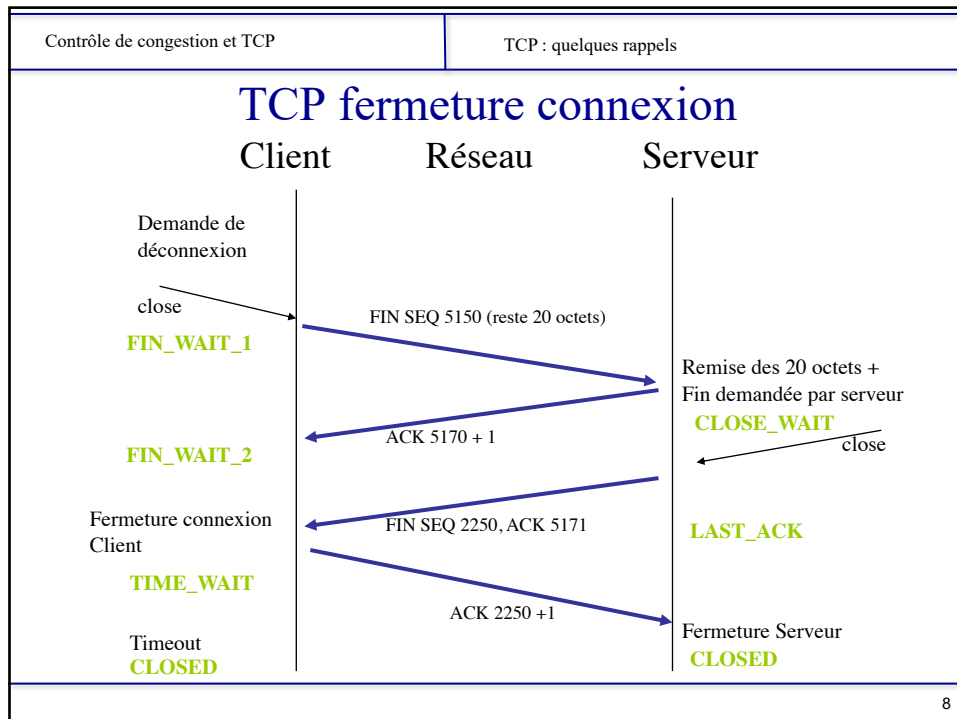
5



6

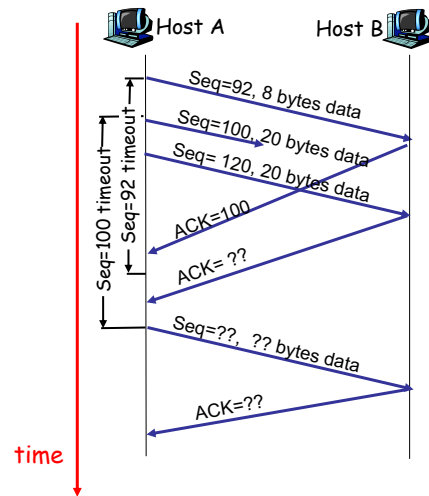


7



8

TCP: scénario de retransmission

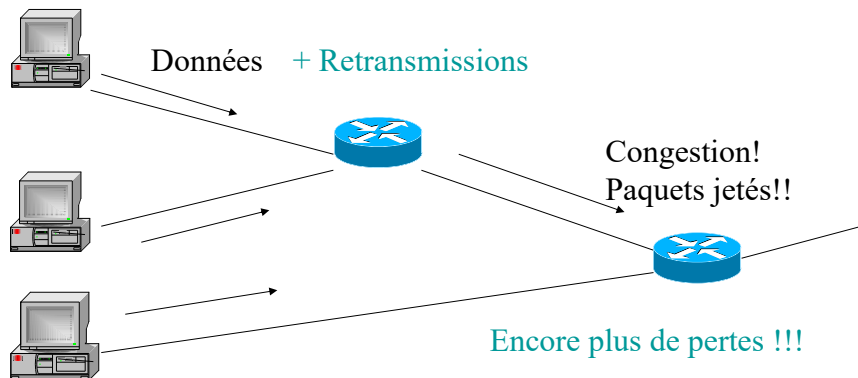


9

9

Congestion Internet - Problématique

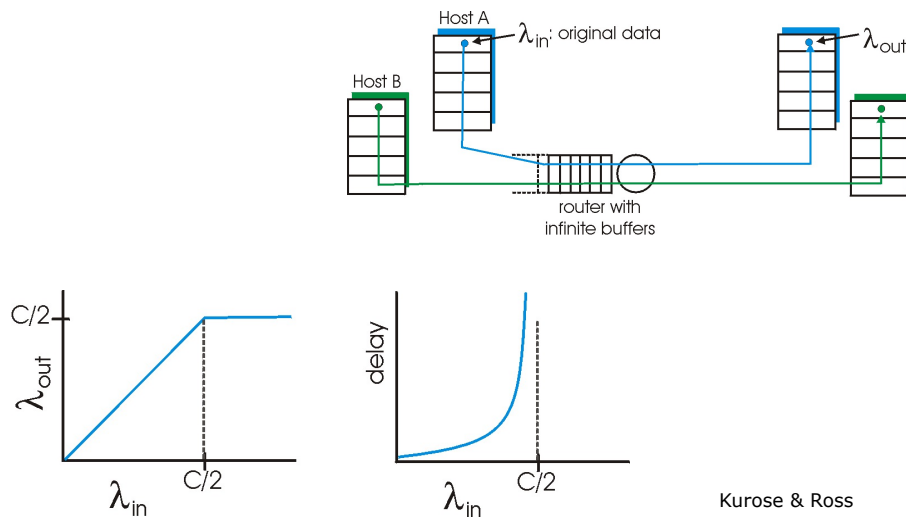
- Dans les années 80, avenir de TCP/IP en question
 - Trop de congestion



10

10

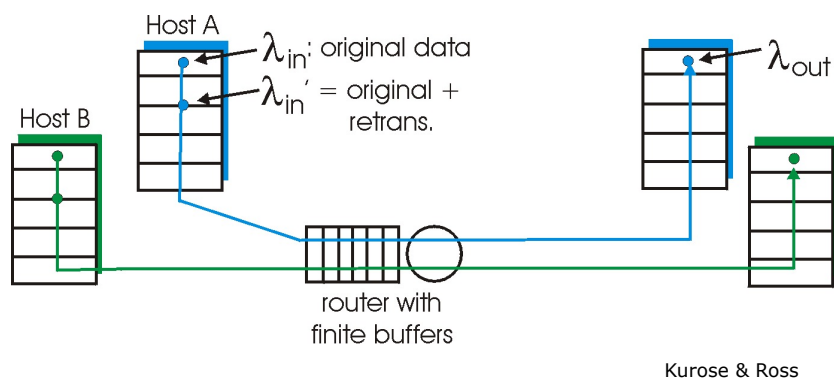
Causes/coûts de la congestion : scénario 1



11

11

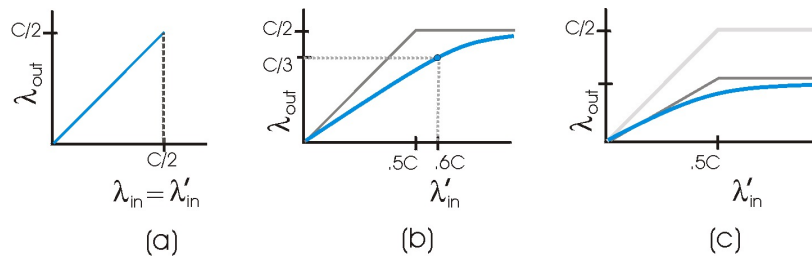
Causes/coûts de la congestion : scénario 2



12

12

Causes/coûts de la congestion : scénario 2

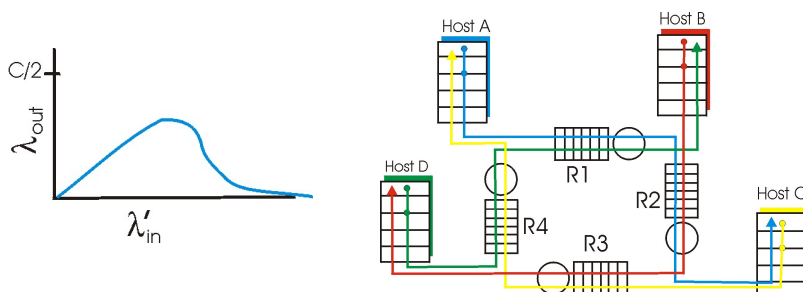


Kurose & Ross

13

13

Causes/coûts de la congestion: scenario 3



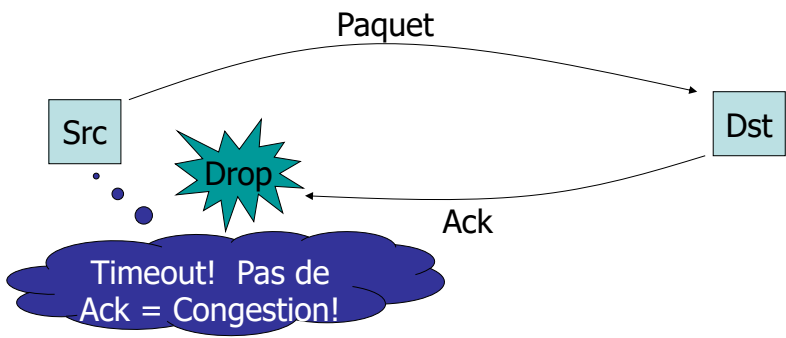
Kurose & Ross

14

14

Contrôle de congestion et TCP	Congestion Internet
• Comment améliorer TCP afin qu'il envoie moins de trafic quand le réseau est congestionné ? – Comment détecter la congestion ? – Comment rendre le flux de données sensible au niveau de congestion ?	
15	

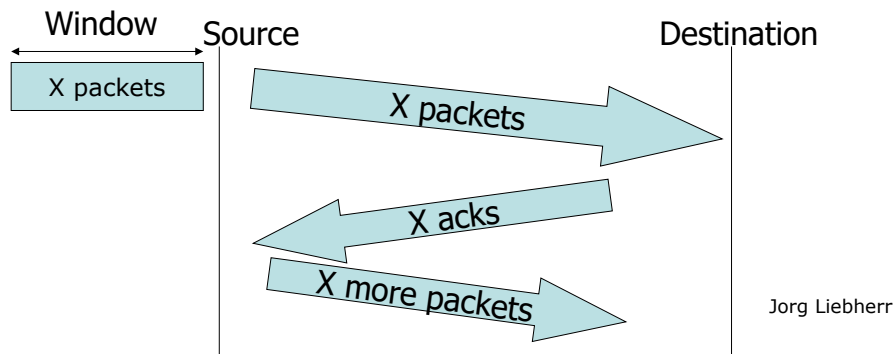
15

Contrôle de congestion et TCP	Congestion Internet
Détection de la congestion • Elimination de paquets = indicateur de congestion ? 	
16	

16

Contrôle de Congestion – l'effet de la taille de la fenêtre

- Noter que la fenêtre de l'émetteur est égale au nombre de paquets en transit dans le réseau

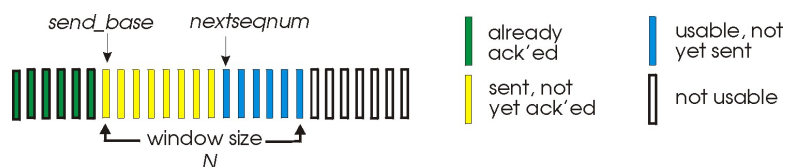


17

17

Contrôle de congestion TCP

- Contrôle de bout-en-bout (pas d'assistance du réseau)
- Débit de transmission limité par taille de fenêtre



Kurose & Ross

18

18

Contrôle de congestion TCP

- w segments, chacun avec MSS octets émis en un RTT:

$$\text{throughput} = \frac{w * \text{MSS}}{\text{RTT}} \text{ Bytes/sec}$$

19

19

Contrôle de congestion TCP

- “sonder” pour la bande passante utilisable:
 - Idéalement, objectif : transmettre aussi vite que possible (fenêtre la + grande possible) sans perte
 - *augmenter* la fenêtre (→ plus de paquets dans le réseau) jusqu’à l’apparition de pertes (congestion)
 - pertes: *diminuer* la fenêtre (→ moins de paquets dans le réseau) et recommencer à sonder (augmenter) à nouveau

20

20

Contrôle de congestion et TCP	Congestion Internet
Contrôle de Congestion • Idée: Concept de la <i>congestion window (cwnd)</i> – fenêtre qui est petite quand la congestion est importante – fenêtre qui est grande quand la congestion est petite • Fenêtre autorisée = minimum (fenêtre récepteur, fenêtre de congestion)	
21	

21

Contrôle de congestion et TCP	Congestion Internet
Contrôle de congestion de Van Jacobson • Van Jacobson a introduit le contrôle de congestion dans TCP en 1988-1989 • La version d'origine de TCP qui implante le contrôle de congestion de Van Jacobson est connue sous le nom de TCP Tahoe	
22	

22

Contrôle de congestion et TCP	TCP Tahoe
Éléments de contrôle de congestion dans TCP Tahoe • AIMD – Additive increase, multiplicative decrease • Slow start • Congestion Avoidance	
23	

23

Contrôle de congestion et TCP	TCP Tahoe
Additive Increase, Multiplicative Decrease • Chaque fois qu'un paquet est détruit (perdu), réduire la taille de la fenêtre de manière significative (multiplicative decrease) – Multiplicative decrease est nécessaire pour éviter la congestion • Si aucune perte n'est observée, augmenter de manière graduelle la taille de la fenêtre (additive increase)	
24	

24

Contrôle de congestion et TCP	TCP Tahoe
Problèmes • Quelle doit être la taille de la fenêtre – Initialement ? – Quand il y a perte de paquet ? • Taille de fenêtre pessimiste? (e.g., 1) – Additive increase est trop lent – Les connexions courtes n'utiliseront jamais la bande passante disponible • Taille de fenêtre optimiste? – Un flux trop important de données peut impliquer un overflow dans les files des routeurs	
25	

25

Contrôle de congestion et TCP	TCP Tahoe
Slow Start • Objectif: déterminer la capacité disponible rapidement • Idée: – Utiliser CongestionThreshold comme une estimation optimiste de CongestionWindow – Commencer avec CongestionWindow = 1 segment (MSS) – doubler CongestionWindow à chaque RTT (incrémenter de 1 segment pour chaque ACK) – Quand CongestionThreshold est atteint utiliser additive increase	
Source Destination	
26	

26

Contrôle de congestion et TCP	TCP Tahoe
-------------------------------	-----------

Slow-start TCP

Algorithme du slow start

initialise: $cwnd = 1MSS$

Pour (chq segment ACKed)

$cwnd++$

Jusqu' à (evt de perte OR

$cwnd \geq ss_threshold$)

The diagram illustrates the slow start process. Host A sends one segment, then two segments, and finally four segments. The time taken for each round trip is labeled as RTT. The number of segments sent doubles each time a segment is acknowledged, showing exponential growth.

- Croissance exponentielle (par RTT) de la taille de fenêtre (pas si lent!)
- Evt de perte: timeout et/ou trois ACKs dupliqués (Tahoe TCP)

Kurose & Ross

27

27

Contrôle de congestion et TCP	TCP Tahoe
-------------------------------	-----------

Illustration du Slow Start

Hypothèse

CongestionThreshold = 8

The graph shows the congestion window size (in segments) on the y-axis (0 to 14) and roundtrip times on the x-axis (t=0 to t=6). The window size increases exponentially until it reaches the congestion threshold of 8 at t=3. After t=3, the window size continues to increase linearly, reaching 12 at t=6.

Roundtrip time	Congestion Window (segments)
t=0	1
t=1	2
t=2	4
t=3	8
t=4	9
t=5	10
t=6	12

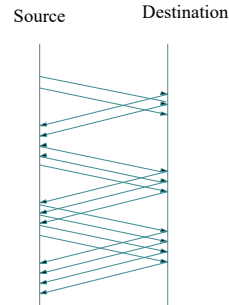
Figures by Jorg Liebeherr

28

28

AIMD + congestion avoidance

- Algorithme :
 - Incrémenter **CongestionWindow** d'un segment par RTT (*linear increase*)
 - Réduire **CongestionThreshold** et **CongestionWindow** (*multiplicative decrease*) quand perte



29

29

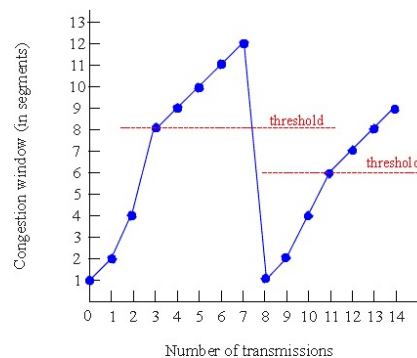
Evitement de congestion TCP Congestion avoidance

Evitement de congestion

```

/* slowstart est terminé */
/* cwnd >= threshold */
Until (evt de perte) {
  chq w segments ACKed:
    cwnd++
}
ss_threshold = cwnd/2
cwnd = 1MSS
effectuer slowstart1
  
```

1: TCP Reno évite slowstart (fast recovery) après 3 ACKs dupliqués



Kurose & Ross

30

30

Contrôle de congestion et TCP	TCP Reno
TCP Reno • TCP Reno (1990) et ses dérivés sont les plus utilisés aujourd'hui • TCP Reno implémente : – Fast Retransmit – Fast Recovery • RFC 2581	
31	

31

Contrôle de congestion et TCP	TCP Reno
Fast Retransmit • Problème : les timers TCP peuvent conduire à des périodes d'inactivité	
<pre> sequenceDiagram participant E as Emetteur participant R as Récepteur E->>R: Packet 1 E->>R: Packet 2 E->>R: Packet 3 E->>R: Packet 4 E->>R: Packet 5 E->>R: Packet 6 R-->>E: ACK 1 R-->>E: ACK 2 R-->>E: ACK 2 R-->>E: ACK 2 R-->>E: ACK 2 R-->>E: ACK 6 E->>R: Retransmit packet 3 R-->>E: ACK 6 </pre>	
32	

32

Contrôle de congestion et TCP	TCP Reno
Fast Retransmit • Envoi d'un acquittement sur réception d'un paquet • Envoi d'une duplication du dernier acquittement quand un paquet est reçu en désordre • Utilisation des acquittements dupliqués pour déclencher des retransmissions – Si 3 (ou plus) d'acquittements dupliqués reçus, le segment est considéré comme perdu – TCP réalise une retransmission du segment manquant sans attendre l'expiration du timer	
33	

33

Contrôle de congestion et TCP	TCP Reno
Fast Recovery • Fast Recovery est implémenté avec Fast Retransmit • Fast Recovery supprime la phase de slow start après retransmission due à un Fast Retransmit • Fast Recovery gère la transmission des données jusqu'à l'arrivée d'un ACK non dupliqué	
34	

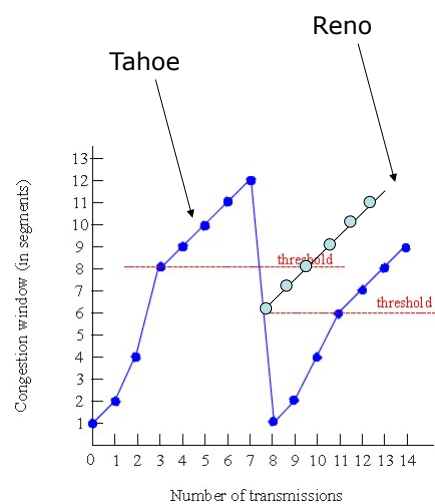
34

Fast Recovery

- Algorithme :
 - Quand le 3ème ACK dupliqué est reçu,
 - positionner le seuil ($ss_threshold$) à la moitié de la *congestion window* courante
 - retransmettre le segment manquant
 - Positionner $cwnd$ à la valeur du $ss_threshold$ + $3 \cdot MSS$
 - Chaque fois qu'un autre ACK dupliqué arrive
 - Incrémenter $cwnd$
 - Transmettre un paquet si possible
 - Quand ACK arrive qui acquitte la nouvelle donnée
 - Positionner $cwnd$ à la valeur de $ss_threshold$

35

35



Kurose & Ross

36

36

Résumé Congestion Control

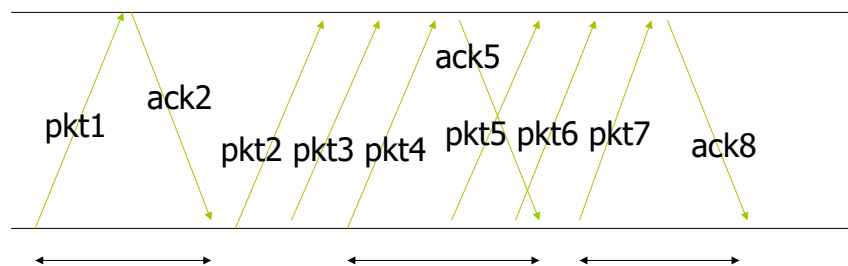
- Essayer d'éviter que la source applique le slow-start
 - Source utilise slow-start au début et sur timeout uniquement
 - Source divise la fenêtre de congestion window de la moitié suite à un fast retransmit
- La perte d'un seul paquet peut être détectée par les mécanismes de fast retransmit/fast recovery algorithm
- La perte de plusieurs paquets consécutifs forcera la source à appliquer le slow start
 - TCP New Reno (RFC3782)

37

37

Tahoe/Reno Timers de retransmission

- Les timers de retransmission reposent sur les calculs du round-trip time (RTT) que TCP effectue



Jorg Liebherr

38

38

Contrôle de congestion et TCP	RTT et TCP
TCP Round Trip Time et Timeout • Comment positionner le Timeout TCP ? • Plus long que RTT – note: RTT va varier • Trop court: timeout prématuré – Retransmissions inutiles • Trop long: réaction lente à la perte de segments	
39	

39

Contrôle de congestion et TCP	RTT et TCP
TCP Round Trip Time et Timeout Comment estimer RTT ? • SampleRTT: temps mesuré entre l'envoi d'un segment et la réception d'un ACK • SampleRTT va varier, besoin d'un RTT moyen, plus lissé – Utilisation de plusieurs mesures récentes, pas uniquement le SampleRTT courant	
40	

40

Contrôle de congestion et TCP	RTT et TCP
TCP Round Trip Time et Timeout • Moyenne fluctuante • L'influence d'un échantillon diminue de façon exponentielle • Valeur typique de α: 0.125 (1/8) • Extrait du rfc 2988 $\text{EstimatedRTT} = (1-\alpha).\text{EstimatedRTT} + \alpha.\text{SampleRTT}$	
41	

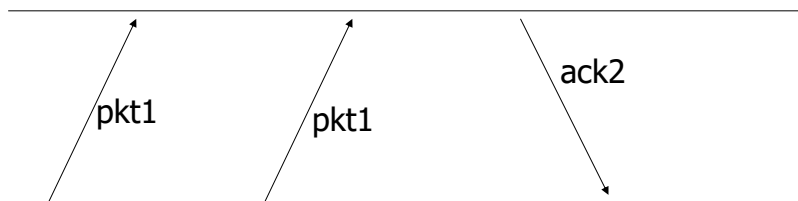
41

Contrôle de congestion et TCP	RTT et TCP
TCP Round Trip Time et Timeout • Valeur du timeout – EstimatedRTT plus “marge de sécurité” – large variation dans EstimatedRTT → marge de sécurité accrue $\text{RetransmissionTimeout ou RTO} = \text{EstimatedRTT} + 4*\text{Deviation}$ $\text{Deviation} = (1-\beta).\text{Deviation} + \beta.* \text{SampleRTT}-\text{EstimatedRTT} $ Avec $\beta = 0.25$	
42	

42

Problème

- Si un segment est retransmis, TCP ne peut pas calculer le RTT
 - Pourquoi ?



43

43

Algorithme de Karn

- Ne pas mettre à jour le RTT sur des segments qui ont été retransmis.
- Quand TCP retransmet, positionner :
 $RTO = \min(2 RTO, 64)$

44

44

Contrôle de congestion et TCP	TCP Vegas
Stratégies de contrôle de congestion • Stratégie TCP – Contrôle de congestion une fois détectée – De manière répétitive, augmente la charge pour trouver le point de congestion puis diminution de la charge quand congestion • Stratégie alternative – Prédire quand la congestion va se produire – Réduire le débit avant que le paquet soit détruit – Deux possibilités : • Orientée routeur : mécanisme RED • Orientée hôte : TCP Vegas	
45	

45

Contrôle de congestion et TCP	TCP Vegas
TCP Vegas • Vegas: Prédit et évite la congestion avant qu'elle se produise • Tahoe, Reno: Réagit à la congestion une fois qu'elle s'est produite • Question: comment prédire la congestion ?	
46	

46

Principe

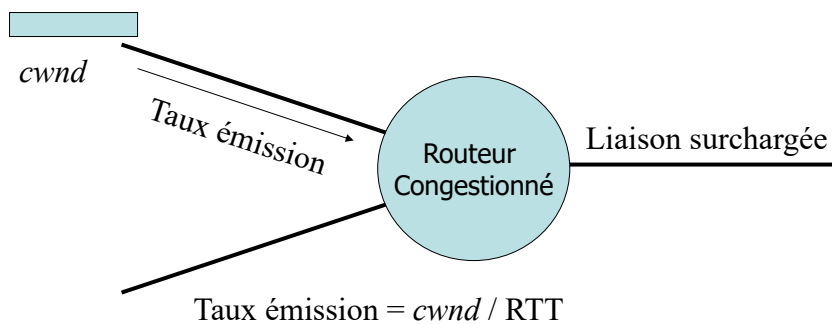
- L. Brakmo, S.O'Malley et L. Peterson (SIGCOMM'94)
- Idée :
 - La source observe si des signes indiquent que les files des routeurs se remplissent et qu'une congestion risque de se produire
 - RTT augmente
 - Le débit d'émission fluctue

47

47

Observation

- L'accumulation de paquets dans le réseau peut se déduire en observant le RTT et débit en émission



Jorg Liebherr

48

48

Contrôle de congestion et TCP	TCP Vegas
Algorithme • Soit BaseRTT le minimum de tous les RTT mesurés (généralement RTT du premier paquet) $\text{ExpectedRate} = \text{CongestionWindow} / \text{BaseRTT}$ • Source calcule ActualRate une fois par RTT	
49	

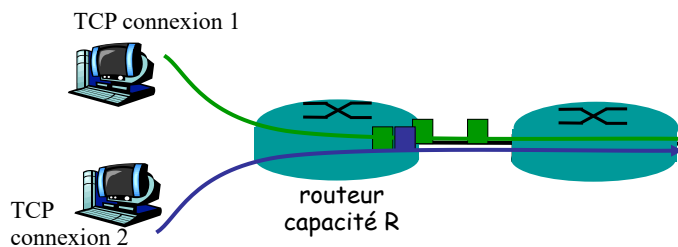
49

Contrôle de congestion et TCP	TCP Vegas
Algorithme • Source compare ActualRate avec ExpectRate <pre> Diff = ExpectedRate - ActualRate if Diff < a augmente CongestionWindow linéairement else if Diff > b décroître CongestionWindow linéairement else laisser CongestionWindow inchangée </pre>	
50	

50

Equité TCP

Si N sessions TCP partagent le même lien goulot d'étranglement, chacun doit avoir $1/N$ de la capacité du lien



Kurose & Ross

51

51

Pourquoi TCP est équitable ?

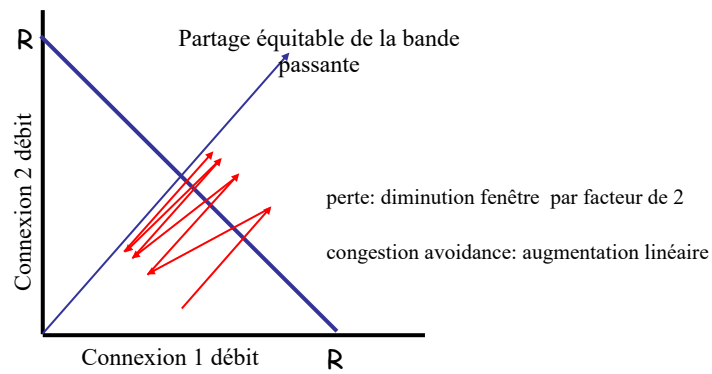
Deux sessions en compétition:

- Augmentation additive donne une pente de 1, lorsque le débit augmente
- Diminution multiplicative diminue le débit proportionnellement

52

52

Pourquoi TCP est équitable ?



53

53

Protocoles de Transport : *Contrôle de Congestion avec TCP*

Rémi Badonnel
Slides d'Isabelle Chrisment

54